Leaper: next training runs

Eight full experiments + one short diagnostic | Planning document | 12 September 2026

Owner requirements

Revised owner decision: decrease the learning rate slowly, with a higher floor. This supersedes the earlier constant-rate instruction and PDF. Use one linear schedule from **0.00015 to 0.00003** across the full 2,031,616-transition horizon for all nine experiments. The floor is ten times the old final rate. No early 508K decay endpoint, restart, or drop below the floor. Preserve global schedule progress on resume; hold the floor during later extensions unless explicitly revised.

Keep all 34 observations unchanged. Leaper must learn body and leg clearance through experience. Do not add clearance sensors, change target-direction coordinates, or supply extra target information. Memory may process previous observations already available to the robot.

Keep static targets, randomized arenas and existing collision physics. Reverse is a separate action experiment. All new runs start from random weights; no pretrained walker is imported. This document does not launch training.

Why this batch

The completed LSTM reached 60.5% final arrival success on 200 development mazes, with a best checkpoint of 63.5%. Final failures were 68 stuck and 11 timeouts. Target memory persists after discovery, yet some recorded episodes moved without collisions and still failed to arrive.

The reward already favors successful arrival, but does not guarantee the policy learns the required actions. Weak time costs, distance feedback, future-reward weighting, recovery actions and memory handling are separate explanations to test. None is established as the sole cause.

Common experiment settings

Setting	Contract
Architecture	Existing additive stateful LSTM, except the explicit-window comparison.
Learning rate	Linear 0.00015 to 0.00003 over 2,031,616 transitions; clamp at the floor afterward.
Budget	Runs 1-8: 2,031,616 transitions each. Run 9: initial 106,496-transition diagnostic.
PPO settings	8 environments x 1,024 steps = 8,192 transitions per rollout; batch 256; 10 epochs. Gamma 0.995 except run 7. Other existing PPO settings remain fixed.
Evaluation	Approximately every 50,000 transitions, rounded to rollout boundaries; final evaluation included. Same 200 development mazes; one training seed per configuration.

The run list

ALL RUNS: LINEAR LEARNING RATE 0.00015 TO 0.00003.
Use the full 2,031,616-step schedule, including for the short diagnostic. Resume at the saved global step; hold 0.00003 after the horizon.

Each row changes only the stated settings relative to run 1. Reward cost values below are positive magnitudes subtracted from reward. Time-cost changes apply only after target discovery; search rewards remain unchanged.

Run	Exact change	Question
1. Control	**LR: linear 0.00015 to 0.00003.** Existing forward-only LSTM. Time cost 0.002; distance coefficient 0.2; gamma 0.995.	Reference under the revised linear schedule.
2. Reverse	**LR: linear 0.00015 to 0.00003.** Signed throttle: negative reverses, zero stops, positive moves forward.	Does backward movement improve recovery and arrival?
3. Moderate urgency	**LR: linear 0.00015 to 0.00003.** Post-discovery time cost 0.01 per step.	Does stronger time pressure reduce moving timeouts?
4. Stronger urgency	**LR: linear 0.00015 to 0.00003.** Post-discovery time cost 0.05 per step.	Does more pressure help, or increase collisions and early failures?
5. Distance feedback	**LR: linear 0.00015 to 0.00003.** Post-discovery distance coefficient 0.25: reward approach and penalize retreat equally.	Does 25% stronger distance feedback improve pursuit?
6. Reverse + urgency	**LR: linear 0.00015 to 0.00003.** Combine run 2 reverse with run 3 time cost of 0.01.	Do recovery freedom and urgency work better together?
7. Longer reward horizon	**LR: linear 0.00015 to 0.00003.** Gamma 0.999 instead of 0.995.	Does stronger weighting of distant outcomes improve pursuit?
8. Explicit memory window	**LR: linear 0.00015 to 0.00003.** 64 consecutive existing observations; reset recurrent state for each window. Match training and inference.	Does finite-window processing improve behavior compared with carried state?
9. Aggressive diagnostic	**LR: linear 0.00015 to 0.00003.** Post-discovery time cost 0.25; initial 106,496-transition budget.	Does very strong time pressure improve behavior or encourage early failure?

Priority and concurrency

Highest priority: runs 1-5. All configurations are independent enough to run simultaneously, subject to a measured CPU/RAM throughput check. With two effective slots, queue pairs 1/2, 3/4, 5/6 and 7/8; schedule diagnostic 9 in the next available slot. Do not assume nine concurrent jobs are faster.

Run 9 is a behavior and stability screen. Extend it only if arrival and failure behavior justify a larger declared budget. A low early score alone does not establish its long-term potential. Preserve all completed and interrupted results.

What the reward changes mean

Distance coefficient: 0.2 to 0.25

Distance reward = coefficient x (distance before movement - distance after movement). This uses straight-line world distance after target discovery. It is not a flat penalty per backward step. Time, collision and arrival rewards are separate.

Movement result	Current: 0.2	Proposed: 0.25
1 unit closer	+0.20	+0.25
1 unit farther away	-0.20	-0.25
Same distance	0	0

Moving away and recovering the same distance cancels before time costs and discounting. Stronger shaping amplifies both approach and retreat feedback; it does not directly punish circling at constant distance. A maximum-speed ten-step retreat covers at most 3.75 units: current distance cost 0.75, proposed 0.9375, plus time costs.

Discount factor: 0.995 to 0.999

Value now = reward now + gamma x next reward + gamma squared x the following reward + ... Higher gamma gives both future rewards and future penalties greater weight. The actual arrival payout remains +25.

Arrival reward timing	Gamma 0.995	Gamma 0.999
Immediately	25.00	25.00
100 steps later	15.14	22.62
500 steps later	2.04	15.16

The table isolates the arrival reward; route costs also contribute. Higher gamma may help connect earlier detour actions with eventual success, but can make value learning harder. It also reduces the discount advantage of arriving sooner, so time costs remain relevant.

Time cost: urgency rather than clearance information

Post-discovery cost per step	Cost of 100 steps	Cost of 500 steps
0.002 (control)	0.2	1
0.01	1	5
0.05	5	25
0.25 (diagnostic)	25	125

These are undiscounted time costs only. The 0.25 diagnostic is 125 times the current rate and can change the task incentive substantially. Measure whether the robot starts ending episodes early rather than reaching the target.

Execution and decision rules

Before launching

1. Recover and verify the intended seeker code revision: the handoff documents were read from Git commit 7689aca because they were absent from the current checkout. Preserve existing workspace changes and archive source hashes for every run.

2. Validate actual optimizer rates at the first, middle and final updates and after resume: 0.00015, 0.00009 and 0.00003. Use collected transitions as the clock, not elapsed time or optimizer minibatches. Verify floor clamping and unchanged schedule horizon on resume.

3. Validate reverse mapping, reward arithmetic and window boundaries. Window inputs must not cross workers or episodes; padding must not become fake experience. Confirm the original 34-feature observation contract remains unchanged.

4. Benchmark concurrent throughput, including evaluations. Keep each run in a separate artifact folder. Record model counts, seeds, packages, initial weights and empty optimizer state.

5. Add separate actor/critic and memory/direct-branch gradient diagnostics and actual weight-update measurements. Verify logging does not change optimizer updates.

Measure outcomes, not reward alone

Primary measure: unassisted target arrival. Also record hidden-only discovery; post-discovery arrival and timeouts; remaining target distance at timeout; stuck/frozen failures by discovery phase; collision rates; pursuit duration with failures included; reverse usage; and reward components separately before and after discovery.

Compare runs at equal training budgets. Report best and final checkpoints separately. A lower collision rate, a shorter failed episode or a higher shaped reward is not enough to claim improvement. The one-seed-per-configuration design screens hypotheses; it does not establish repeatability across training seeds.

Keep the 1,000 reserved confirmation mazes (120000-120999) untouched until candidate and control checkpoints are frozen. Development mazes remain 30000-30199. Do not replace the browser production model during this research batch.

Launch and progress reporting

Announce each launch with its hypothesis, architecture/parameter count, exact changes, random initialization, budget and revised linear learning-rate schedule. Provide five-minute updates during active training. No launch or monitoring restart has been performed for this PDF request.

Evidence and scope

Plan based on the owner decisions in this conversation; SEEKER_HANDOFF.md, SEEKER_SCRATCH_RUN_LOG_2026-09-09.md and SEEKER_FINDINGS_AND_NEXT_EXPERIMENTS_2026-09-10.md recovered from commit 7689aca; and saved campaign artifacts under rl_artifacts/seeker_scratch_lr_20260909/.

Key evidence: residual_lstm_extension_2m/dev_2031616_summary.json; dev_2007040_episodes.json; gradient_audit/audit.json. The current reward and action definitions were checked in rl_environment.py at the documented revision. New configurations above are prospective, not measured improvements.

Learning-rate decision

Verified previous LSTM schedule

Stage	Learning rate	Reduction
Start to 507,904 steps	0.00015 to 0.000015	10-fold lower; 90% reduction
507,904 to 2,031,616 steps	0.000015 to 0.000003	5-fold lower; 80% further reduction
Entire run	0.00015 to 0.000003	50-fold lower; 98% reduction

Both endpoints were verified directly in the saved optimizer states of model_507904.zip and model_2031616.zip. This is not only a configured schedule. The old first stage exhausted its 90% decrease after just one quarter of the eventual total budget.

Recommendation: full-horizon linear decay

LR(t) = 0.00003 + 0.00012 x (1 - p), where p is collected transitions / 2,031,616, clamped between 0 and 1. This starts at the previously used rate and ends at a floor ten times higher than before. The new total reduction is 80%, rather than 98%; the decline per transition is 4.5 times gentler than the old first stage.

A five-times-higher floor would be 0.000015. Choose 0.00003 as the first candidate to preserve more late learning. This is an evidence-informed setting to test, not a proven optimum.

Budget used	Old two-stage	New linear	Same-endpoint cosine
0%	0.000150	0.000150	0.000150
25%	0.000015	0.000120	0.00013243
50%	0.000011	0.000090	0.000090
75%	0.000007	0.000060	0.00004757
100%	0.000003	0.000030	0.000030

Cosine starts gently, falls fastest around the midpoint and flattens near its floor. With the same endpoints it is higher than linear in the first half and lower in the second. Linear is the simpler first choice for the stated late-learning concern. Cosine formula: floor + 0.5 x (start - floor) x (1 + cos(pi x p)); no restarts. Definition checked against official PyTorch CosineAnnealingLR documentation.

Evidence limits and execution

Late audit medians: combined pre-clipping gradient norm 179.50, learning rate 0.00000629 and approximate KL 0.000862. Gradients remained nonzero and policy changes became smaller; this does not prove the rate caused the plateau or that all remaining gradients help action learning. Record separate actor/critic gradients and actual weight changes alongside arrival.

Run 9 uses the same 2.03M schedule even when screened at 106,496 steps: its rate there is about 0.00014371, not the floor. If extended, preserve that clock and optimizer state. Do not silently spread different schedule shapes across the behavior comparisons. A later cosine comparison should change only the schedule on the control model. No training code or active job was modified.

Reference: https://docs.pytorch.org/docs/main/generated/torch.optim.lr_scheduler.CosineAnnealingLR.html